

1ª AVALIAÇÃO
Trabalho de Implementação — Jogo da Força

1. Contextualização

O Jogo da Força é um jogo tradicional de palavras no qual o jogador deve descobrir uma palavra oculta por meio da escolha sucessiva de letras. A cada tentativa incorreta, uma nova parte da representação da força é exibida. O jogador vence quando identifica todas as letras da palavra antes de atingir o limite máximo de erros permitido.

Embora apresente regras simples e amplamente conhecidas, sua implementação envolve diferentes problemas de programação, incluindo manipulação de strings, coleções, gerenciamento de estado, sorteio de elementos, validação de regras de negócio, leitura de dados estruturados e atualização dinâmica da interface.

A proposta deste trabalho é desenvolver, utilizando **Flutter e Dart**, uma versão completa do Jogo da Força para dispositivos móveis.

A aplicação deverá funcionar integralmente de forma **offline**. As palavras, categorias, níveis de dificuldade, dicas e demais informações necessárias ao funcionamento do jogo serão fornecidas juntamente com este enunciado em um **arquivo JSON**, que deverá ser incorporado aos assets da aplicação.

Não deverá ser utilizado banco de dados, serviço web, API externa ou qualquer outro mecanismo de armazenamento ou comunicação remota.

2. Requisitos Funcionais

RF01 — Início do jogo

O aplicativo deverá permitir ao usuário iniciar uma nova rodada selecionando:

- uma categoria de palavras;
- um nível de dificuldade.

As categorias e os níveis disponíveis deverão ser obtidos a partir dos dados fornecidos no arquivo JSON.

Antes do início da rodada, o aplicativo deverá apresentar claramente as opções selecionadas.

RF02 — Categorias e níveis de dificuldade

As palavras deverão estar organizadas em categorias e níveis de dificuldade conforme os dados fornecidos.

A aplicação deverá:

- apresentar apenas categorias existentes no conjunto de dados;
- permitir selecionar o nível de dificuldade;
- selecionar somente palavras compatíveis com a categoria e o nível escolhidos;
- utilizar o arquivo JSON fornecido como fonte de dados para palavras e dicas.

A equipe não deverá alterar o significado das categorias ou dos níveis definidos no conjunto de dados.

RF03 — Sorteio da palavra

Ao iniciar uma rodada, uma palavra compatível com a categoria e o nível escolhidos deverá ser selecionada aleatoriamente.

A palavra selecionada deverá permanecer oculta inicialmente, sendo mostradas apenas as posições correspondentes às suas letras.

Exemplo:

Palavra: COMPUTADOR

Exibição inicial:

RF04 — Não repetição de palavras

Uma palavra já utilizada não poderá ser sorteada novamente durante a mesma sessão da aplicação enquanto houver outras palavras disponíveis para a combinação de categoria e nível selecionada.

A aplicação deverá controlar as palavras já utilizadas durante a sessão.

Quando todas as palavras disponíveis para determinada combinação de categoria e nível tiverem sido utilizadas, a aplicação deverá informar essa condição ao usuário e tratá-la adequadamente.

Não é permitido repetir silenciosamente palavras já apresentadas.

RF05 — Tentativa de letras

O jogador deverá realizar suas tentativas selecionando letras do alfabeto disponibilizadas pela própria interface da aplicação.

Para cada letra escolhida:

- se a letra estiver presente na palavra, todas as suas ocorrências deverão ser reveladas;
- se a letra não estiver presente, deverá ser contabilizado um erro;
- uma letra já escolhida não poderá produzir uma nova tentativa;
- a interface deverá permitir identificar quais letras já foram utilizadas.

A equipe poderá definir livremente a organização visual do teclado ou do conjunto de letras.

RF06 — Tratamento de acentuação

A comparação entre a letra escolhida pelo jogador e as letras da palavra deverá **ignorar acentuação**.

Assim:

- A deverá corresponder a A , Á , À , Â e Ã ;
- E deverá corresponder a E , É e Ê ;
- I deverá corresponder a I e Í ;
- O deverá corresponder a O , Ó , Ô e Õ ;
- U deverá corresponder a U e Ú .

A palavra exibida ao usuário deverá, entretanto, **preservar sua grafia original**.

Exemplo:

Palavra: CORAÇÃO

Tentativa: A

Nesse caso, a ocorrência correspondente a ã deverá ser revelada, permanecendo exibida como ã .

A normalização utilizada exclusivamente para comparação não deverá modificar a representação original da palavra.

RF07 — Controle de erros

A aplicação deverá definir um número máximo de erros permitidos para cada rodada.

Cada tentativa incorreta deverá:

- incrementar o número de erros;
- atualizar a representação visual da força;
- atualizar as informações apresentadas ao jogador.

O jogador perderá a rodada quando atingir o número máximo de erros permitido.

RF08 — Representação gráfica progressiva da força

O aplicativo deverá apresentar uma representação gráfica progressiva da força e do boneco.

A representação deverá evoluir conforme o número de erros cometidos.

O boneco deverá ser formado, no mínimo, pelas seguintes partes:

1. cabeça;
2. tronco;
3. braço esquerdo;
4. braço direito;
5. perna esquerda;
6. perna direita.

A estrutura da força poderá estar presente desde o início da rodada ou fazer parte da sequência visual adotada pela equipe, desde que a progressão dos erros seja claramente perceptível.

Cada novo erro deverá produzir uma alteração visual correspondente.

A equipe poderá utilizar:

- imagens armazenadas nos assets;
- composição de widgets;
- ícones;
- desenho programático.

Não será exigida a utilização de `CustomPainter`.

O limite máximo de erros deverá ser compatível com a quantidade de estados gráficos implementados.

RF09 — Sistema de dicas

Cada palavra possuirá uma ou mais dicas no arquivo JSON fornecido.

Durante a rodada, o jogador deverá poder solicitar uma dica relacionada à palavra atual.

O sistema deverá:

- apresentar a dica ao jogador;
- controlar se uma dica já foi utilizada naquela rodada;
- contabilizar sua utilização nas estatísticas da sessão.

A consulta repetida à mesma dica durante uma rodada não deverá produzir contabilizações adicionais.

RF10 — Condição de vitória

O jogador vencerá a rodada quando todas as letras da palavra tiverem sido corretamente identificadas antes de atingir o limite máximo de erros.

Ao final de uma rodada vencida, a aplicação deverá apresentar, no mínimo:

- a palavra completa;
 - indicação de vitória;
 - quantidade de erros;
 - indicação de utilização ou não de dica.
-

RF11 — Condição de derrota

O jogador perderá a rodada quando atingir o limite máximo de erros.

Nesse caso, a aplicação deverá:

- informar explicitamente a derrota;
- revelar a palavra correta;
- apresentar a representação final da força;
- permitir iniciar uma nova rodada.

RF12 — Nova rodada

Após o encerramento de uma rodada, o usuário deverá poder iniciar outra.

A aplicação deverá manter durante a sessão:

- as estatísticas acumuladas;
- o controle das palavras já utilizadas.

Também deverá ser possível retornar à seleção de categoria e nível.

RF13 — Estatísticas da sessão

A aplicação deverá manter estatísticas enquanto permanecer em execução.

Deverão ser apresentadas, no mínimo:

- número total de rodadas realizadas;
- número e percentual de vitórias;
- número e percentual de derrotas;
- quantidade total de erros cometidos;
- quantidade de dicas utilizadas.

Os percentuais deverão ser calculados a partir das rodadas concluídas.

Os dados poderão ser reinicializados quando a aplicação for encerrada.

Não deverá ser implementada persistência dessas estatísticas em banco de dados ou serviço externo.

A interface deverá possuir uma forma clara de consultar as estatísticas da sessão.

RF14 — Identificação dos membros da equipe

A aplicação deverá possuir uma funcionalidade que permita visualizar os nomes completos dos integrantes da equipe responsável pelo desenvolvimento.

Essa informação poderá ser apresentada, por exemplo, em:

- tela específica;
- opção "Sobre";
- caixa de diálogo;
- seção equivalente.

Os nomes deverão estar acessíveis ao usuário a partir da interface da aplicação.

3. Dados fornecidos em JSON

As informações necessárias para execução do jogo serão disponibilizadas juntamente com este enunciado em um arquivo JSON.

O arquivo conterá informações relacionadas a:

- palavra;
- categoria;
- nível de dificuldade;

- dica ou dicas associadas.

A estrutura seguirá o formato geral abaixo:

```
{
  "words": [
    {
      "word": "CORAÇÃO",
      "category": "humanBody",
      "difficulty": "medium",
      "hints": ["Órgão do sistema circulatório"]
    },
    {
      "word": "COMPUTADOR",
      "category": "technology",
      "difficulty": "easy",
      "hints": ["Dispositivo utilizado para processamento de dados"]
    }
  ]
}
```

Campos utilizados para representar conceitos que possuam um **conjunto fechado e previamente definido de valores** deverão ser obrigatoriamente modelados no código Dart por meio de tipos enumerados (`enum`).

Esses valores não deverão ser utilizados internamente apenas como `String` quando representarem categorias, níveis, estados ou outros conceitos discretos previamente conhecidos.

Por exemplo, considerando que o campo `category` possua valores como `humanBody`, `technology`, `sports` e `animals`, sua representação no domínio deverá utilizar um tipo enumerado equivalente:

```
enum WordCategory {
  humanBody,
  technology,
  sports,
  animals,
}
```

Da mesma forma, o campo `difficulty` deverá ser representado por um tipo enumerado:

```
enum Difficulty {
  easy,
  medium,
  hard,
}
```

A conversão dos valores armazenados no JSON para os respectivos tipos enumerados deverá ocorrer durante o processo de desserialização.

Essa exigência aplica-se a todos os atributos do domínio que representem conjuntos fechados de valores, e não apenas aos campos apresentados nos exemplos anteriores.

O arquivo JSON deverá ser incluído no projeto como um **asset da aplicação**, sendo declarado no `pubspec.yaml` .

Exemplo:

```
flutter:
  assets:
    - assets/data/words.json
```

O conteúdo deverá ser carregado em tempo de execução. Uma possibilidade é utilizar `rootBundle` :

```
import 'package:flutter/services.dart';

final jsonString =
  await rootBundle.loadString('assets/data/words.json');
```

A partir da `String` carregada, a equipe deverá realizar a desserialização do JSON e convertê-lo em objetos Dart adequadamente modelados.

A equipe deverá implementar:

1. carregamento do asset;
2. desserialização do JSON;
3. conversão dos dados para objetos Dart;
4. conversão dos campos com valores predefinidos para os respectivos tipos enumerados;
5. representação dos dados por meio de classes adequadas;
6. tratamento apropriado de erros ou inconsistências durante o carregamento.

Não é permitido copiar manualmente o conteúdo do arquivo JSON para listas, mapas, constantes ou objetos definidos diretamente no código-fonte.

A aplicação deverá obrigatoriamente carregar e processar, em tempo de execução, o arquivo JSON fornecido com a atividade.

Implementações que reproduzam os dados do arquivo diretamente no código-fonte, evitando seu carregamento e desserialização, serão consideradas como **não atendimento deste requisito**, ainda que apresentem comportamento funcional equivalente.

4. Regras de negócio obrigatórias

As seguintes regras deverão ser satisfeitas independentemente das decisões de interface adotadas pela equipe:

1. uma letra correta deverá revelar todas as suas ocorrências na palavra;
2. uma letra incorreta deverá gerar somente um erro;
3. selecionar novamente uma letra já utilizada não deverá produzir novo erro;
4. a comparação entre letras deverá ignorar acentuação;
5. a grafia original da palavra deverá ser preservada na interface;
6. palavras não deverão ser repetidas enquanto houver palavras ainda não utilizadas na mesma categoria e nível;
7. vitória e derrota são estados mutuamente exclusivos;
8. nenhuma tentativa deverá ser aceita depois do encerramento da rodada;
9. a utilização da dica deverá ser registrada no máximo uma vez por rodada;
10. as estatísticas deverão refletir corretamente todas as rodadas concluídas.

5. Requisitos Não Funcionais

RNF01 — Usabilidade

A interface deverá:

- ser intuitiva;
- apresentar claramente o estado atual da rodada;
- distinguir letras disponíveis das já utilizadas;
- fornecer feedback visual após cada tentativa;
- deixar evidentes as condições de vitória e derrota.

RNF02 — Interface responsiva

A aplicação deverá funcionar adequadamente em diferentes dimensões de tela de dispositivos Android.

Não deverão ocorrer:

- elementos sobrepostos;
- textos cortados;
- controles inacessíveis;
- erros de overflow;
- dependência de uma resolução específica.

RNF03 — Qualidade visual

Cada equipe possui liberdade para definir a identidade visual da aplicação.

Serão considerados na avaliação:

- organização;
- legibilidade;
- consistência visual;
- hierarquia das informações;
- uso adequado dos componentes do Flutter;
- qualidade geral da experiência oferecida.

Recursos gráficos não deverão comprometer a usabilidade da aplicação.

RNF04 — Idioma, nomenclatura e estilo de código

A interface da aplicação, a documentação textual e os demais conteúdos destinados à leitura humana deverão estar em **português**, salvo quando o próprio contexto técnico exigir outra forma.

Deverão estar em **inglês** os elementos técnicos de identificação utilizados no desenvolvimento, incluindo:

- classes;
- métodos e funções;
- variáveis e propriedades;
- tipos enumerados e respectivos valores;
- identificadores internos;
- nomes de arquivos de código;
- nomes de diretórios criados pela equipe;
- nomes técnicos de assets;
- nomes de branches;
- nome do repositório;
- componentes técnicos de URLs.

Exemplos:

```
hangman-game
word_repository.dart
game_screen.dart
assets/data/words.json
feature/game
```

Palavras do jogo, dicas, mensagens da interface, documentação e demais textos destinados a pessoas poderão permanecer em português.

A codificação deverá seguir as recomendações do **Effective Dart**:

<https://dart.dev/effective-dart>

RNF05 — Qualidade do código

O projeto deverá:

- utilizar código organizado e legível;
- evitar duplicação desnecessária;
- utilizar nomes expressivos;

- decompor adequadamente widgets e responsabilidades;
- empregar `const` sempre que pertinente;
- separar adequadamente lógica de negócio e interface.

O comando:

```
flutter analyze
```

deverá ser executado sem erros ou advertências provenientes do código desenvolvido pela equipe.

O código deverá estar formatado utilizando:

```
dart format .
```

RNF06 — Funcionamento offline

A aplicação deverá funcionar integralmente sem acesso à Internet.

Não poderão ser necessários serviços externos para iniciar, executar ou utilizar qualquer funcionalidade obrigatória.

6. Modelagem e organização da solução

As regras do jogo não deverão ficar integralmente acopladas à interface gráfica.

Espera-se separação adequada entre:

- representação dos dados;
- regras do jogo;
- estado da rodada;
- estatísticas da sessão;
- carregamento dos dados;
- widgets responsáveis pela apresentação.

Classes e tipos enumerados deverão ser utilizados sempre que representarem adequadamente conceitos do domínio, tais como:

- palavra;
- categoria;
- dificuldade;
- estado da rodada;
- estatísticas.

A organização arquitetural fica a critério da equipe, desde que resulte em código organizado, compreensível, de fácil manutenção e coerente com os critérios estabelecidos neste enunciado.

7. Restrições técnicas

Não deverão ser utilizados:

- bancos de dados;
- Firebase ou serviços equivalentes;
- APIs REST;
- APIs externas;
- acesso à Internet;
- autenticação de usuários;
- armazenamento remoto;
- serviços de backend.

Os dados necessários ao jogo deverão estar contidos no arquivo JSON fornecido e nos demais recursos locais do aplicativo.

Pacotes externos somente deverão ser utilizados quando tecnicamente justificados.

A utilização de pacote que implemente diretamente as regras centrais do Jogo da Força poderá resultar em desconsideração dos respectivos requisitos na avaliação.

8. Funcionalidades e telas mínimas

A organização das telas é uma decisão de projeto da equipe. Entretanto, a aplicação deverá fornecer acesso claro às seguintes funcionalidades.

8.1. Configuração da partida

Deverá permitir:

- selecionar categoria;
- selecionar dificuldade;
- iniciar a rodada.

8.2. Tela principal do jogo

Deverá apresentar, no mínimo:

- categoria;
- dificuldade;
- palavra parcialmente revelada;
- letras disponíveis;
- letras já utilizadas;
- representação da força;
- quantidade ou progresso dos erros;
- acesso à dica.

8.3. Resultado da rodada

Deverá indicar:

- vitória ou derrota;
- palavra correta;
- quantidade de erros;
- utilização de dica;
- possibilidade de iniciar nova rodada.

8.4. Estatísticas

Deverá apresentar as estatísticas acumuladas durante a sessão.

8.5. Identificação da equipe

Deverá permitir consultar os nomes completos dos integrantes responsáveis pelo desenvolvimento da aplicação.

9. Composição da equipe

- A implementação deverá ser realizada em equipes de **4 (quatro) estudantes**.
- Todos os integrantes são responsáveis pelo conhecimento integral da solução entregue.
- **Cada integrante será questionado individualmente** durante a sessão de apresentação da equipe, inclusive sobre partes do projeto implementadas originalmente por outros integrantes.
- Todos os membros deverão ser capazes de explicar as principais decisões de projeto, regras de negócio, modelagem, arquitetura e código-fonte.
- A divisão interna de tarefas não elimina a responsabilidade de cada integrante pelo conhecimento da solução completa.
- O histórico do repositório deverá demonstrar participação efetiva de todos os membros.
- A participação de todos os integrantes na sessão de apresentação é **obrigatória**.

A ausência de um integrante na sessão de apresentação implicará **pontuação 0 para esse integrante na composição da nota da apresentação da equipe**.

Essa pontuação zero será considerada normalmente no cálculo da média da apresentação, não implicando, por si só, nota zero para toda a equipe ou para o trabalho completo.

10. Documentação do projeto

O projeto deverá possuir um arquivo `README.md` compatível com o que normalmente se espera de um repositório público de software.

No início do `README.md` deverão constar, obrigatoriamente, os nomes completos de todos os integrantes da equipe.

Após essa identificação, o documento deverá conter, no mínimo:

- nome do projeto;
- breve descrição;
- principais funcionalidades;
- tecnologias utilizadas;
- instruções para obtenção e execução do projeto;
- eventuais dependências adicionais;
- estrutura geral da aplicação, quando pertinente.

A ausência da identificação dos integrantes no início do `README.md` caracterizará descumprimento deste requisito.

11. Repositório Git

O projeto deverá ser mantido em um repositório **público** no GitHub.

O repositório deverá:

- possuir `main` como branch principal;
- estar publicamente acessível no momento da entrega e durante todo o período de avaliação;
- apresentar histórico de commits coerente com o processo de desenvolvimento;
- possuir commits realizados por **todos os integrantes da equipe**;
- permitir identificar participação efetiva dos membros ao longo do desenvolvimento;
- conter o arquivo `README.md` exigido neste enunciado;
- conter apenas arquivos necessários ao desenvolvimento, compilação e execução do projeto.

A participação de todos os integrantes por meio de commits identificáveis no histórico do Git constitui **critério de avaliação**.

Os commits deverão representar contribuições efetivas. A criação artificial de commits apenas para registrar participação poderá ser desconsiderada.

O repositório não deverá conter artefatos gerados pelo processo de compilação, empacotamento ou execução nem arquivos que normalmente devam ser excluídos pelo `.gitignore`.

Isso inclui, quando aplicável:

- diretórios de build;
- arquivos APK;
- Android App Bundles (`.aab`);
- executáveis ou outros binários compilados;
- caches;
- arquivos temporários;
- arquivos intermediários produzidos pelas ferramentas;
- configurações locais de IDE que não façam parte do projeto;
- outros artefatos que possam ser reproduzidos a partir do código-fonte.

O `.gitignore` deverá estar adequadamente configurado para um projeto Flutter.

A inexistência do repositório público ou a impossibilidade de acesso ao endereço informado poderá inviabilizar a avaliação da entrega.

12. Entrega

A entrega deverá ser realizada exclusivamente pelo **SIGAA** até o dia **12/10/2026**.

Deverá ser submetido:

1. o link para o repositório público do projeto no GitHub;
2. imediatamente após o link, um comentário contendo os nomes completos dos integrantes da equipe.

Exemplo:

`https://github.com/username/hangman-game`

Integrantes:

- Nome completo do integrante 1
- Nome completo do integrante 2
- Nome completo do integrante 3
- Nome completo do integrante 4

O nome do repositório e demais elementos técnicos utilizados na URL deverão seguir a convenção de uso do inglês estabelecida neste enunciado.

O repositório informado deverá conter todos os arquivos necessários para obtenção, compilação e execução da aplicação, incluindo:

- código-fonte;
- `pubspec.yaml` ;
- arquivo JSON fornecido para a atividade;
- assets necessários;
- `README.md` ;
- arquivos de configuração indispensáveis ao projeto.

O repositório deverá conter o **código-fonte e os recursos necessários para reproduzir a aplicação**, e não os produtos gerados pelo processo de build.

Portanto, não deverão ser incluídos APKs, Android App Bundles, diretórios de build, binários, caches ou outros artefatos gerados que possam ser reconstruídos a partir do código-fonte.

A aplicação deverá poder ser obtida e executada a partir do repositório entregue sem necessidade de serviços externos.

13. Sistema de avaliação

A avaliação será realizada em **duas etapas**, cada uma contribuindo com até **5,0 pontos** para a nota final:

1. **entrega do código e avaliação da solução pelo professor** — até **5,0 pontos**;
2. **sessão de apresentação, demonstração e arguição da equipe** — até **5,0 pontos**.

A nota final do trabalho será expressa na escala de **0 a 10** e calculada da seguinte forma:

Nota final = nota da entrega + nota da apresentação

Portanto:

- a **Etapla 1** poderá contribuir com até **5,0 pontos**;
- a **Etapla 2** poderá contribuir com até **5,0 pontos**;
- a soma das duas etapas determinará a nota final, limitada a **10,0 pontos**.

13.1. Escala de avaliação

Cada requisito, critério, resposta ou desempenho individual analisado receberá uma pontuação entre **0 e 5**, conforme o grau de atendimento:

Pontuação	Grau de atendimento
0	Não atende.
1 a 4	Atende parcialmente, em grau proporcional à correção, completude e qualidade demonstradas.
5	Atende completamente.

13.2. Etapa 1 — Entrega e avaliação da solução — até 5,0 pontos

Após a entrega pelo SIGAA, o professor avaliará o repositório e a aplicação submetida.

Cada requisito funcional, requisito não funcional e critério técnico aplicável será avaliado individualmente com pontuação entre **0 e 5**.

A **nota da entrega** será determinada pela média das pontuações atribuídas aos itens avaliados, resultando em uma nota entre **0 e 5**.

Serão considerados, entre outros:

- correção funcional;
- atendimento às regras de negócio;
- tratamento de situações de contorno;
- carregamento e modelagem dos dados JSON;
- uso apropriado de tipos enumerados;
- qualidade e organização do código;
- interface, usabilidade e responsividade;
- modelagem e arquitetura;
- documentação;
- organização do repositório;
- histórico de commits;
- participação dos integrantes no processo de desenvolvimento.

A nota obtida nesta etapa será comum a todos os integrantes da equipe e contribuirá com até **5,0 pontos** para a nota final.

Exemplo

Se as avaliações dos requisitos e critérios da entrega resultarem em média **4,2**:

Nota da entrega = 4,2 pontos

13.3. Qualidade e organização do código

Na avaliação da qualidade do código serão considerados:

- clareza;
- legibilidade;
- simplicidade;
- coesão;
- nomenclatura;
- aderência ao Effective Dart;
- ausência de duplicação desnecessária;
- decomposição adequada de widgets e responsabilidades;
- utilização apropriada dos recursos de Dart e Flutter;
- análise estática sem problemas;
- aplicação coerente dos princípios de **Clean Code**.

Espera-se código com responsabilidades claras, nomes expressivos, métodos e funções com propósito bem definido, baixa duplicação e ausência de complexidade desnecessária.

13.4. Modelagem e arquitetura

Será considerada a qualidade da representação dos conceitos do domínio e da organização estrutural da aplicação.

Serão avaliados, entre outros aspectos:

- modelagem adequada dos conceitos do domínio;
- uso apropriado de classes e tipos enumerados;
- separação entre interface, estado, regras de negócio e acesso aos dados locais;
- dependências coerentes entre os componentes;
- baixo acoplamento;
- alta coesão;
- aplicação pertinente dos princípios **SOLID**;
- adoção de princípios compatíveis com **Clean Architecture**, especialmente separação de responsabilidades e direção adequada das dependências.

Não é exigida a reprodução rígida de uma arquitetura específica nem a criação artificial de camadas sem necessidade.

Entretanto, soluções excessivamente acopladas, nas quais regras de negócio, carregamento de dados e interface estejam concentrados nos mesmos componentes, serão penalizadas.

13.5. Processo de desenvolvimento e repositório

Serão considerados:

- histórico de desenvolvimento;
- existência de commits de todos os integrantes;
- distribuição coerente das contribuições;
- qualidade e clareza das mensagens de commit;
- consistência da evolução do projeto;
- qualidade e organização do repositório;
- uso adequado do `.gitignore`;
- ausência de artefatos gerados indevidamente versionados;
- qualidade da documentação;
- conformidade do `README.md` com o enunciado.

A mera existência de commits em nome de um integrante não comprova, isoladamente, participação efetiva. O conteúdo das contribuições poderá ser considerado durante a avaliação.

13.6. Etapa 2 — Sessão de apresentação, demonstração e arguição — até 5,0 pontos

A sessão de apresentação das equipes será realizada no dia **13/10/2026**.

Cada equipe participará de uma sessão com o professor.

A sessão terá como objetivos:

1. demonstrar o funcionamento da solução entregue;
2. verificar o domínio técnico dos integrantes sobre a solução desenvolvida.

A presença de todos os integrantes é **obrigatória**.

Durante a sessão, **cada integrante receberá individualmente uma pontuação entre 0 e 5**, de acordo com o domínio demonstrado durante a apresentação e a arguição.

Cada estudante poderá ser questionado sobre qualquer parte relevante da solução, independentemente de quem tenha realizado originalmente sua implementação.

Poderão ser avaliados:

- compreensão das regras do jogo;
- compreensão da modelagem adotada;
- interpretação do código-fonte;
- justificativa das decisões técnicas;
- fluxo de dados;
- gerenciamento de estado;
- carregamento e desserialização do JSON;
- conversão de valores para tipos enumerados;
- normalização utilizada para ignorar acentuação;
- cálculo das estatísticas;
- organização arquitetural;
- capacidade de localizar funcionalidades no projeto;
- capacidade de explicar ou modificar pequenos trechos da implementação.

Ausência de integrante

O integrante que **não comparecer à sessão de apresentação receberá pontuação 0** na sua participação individual.

Essa pontuação será utilizada normalmente no cálculo da média da equipe para a Etapa 2.

A ausência de um integrante, portanto, **não atribui automaticamente nota zero ao trabalho nem à apresentação da equipe**, mas reduz a média da apresentação em razão da pontuação zero atribuída ao membro ausente.

Cálculo da nota da apresentação

A **nota da apresentação da equipe** será calculada pela média aritmética das pontuações individuais de todos os seus integrantes, incluindo pontuação zero para integrantes ausentes.

O resultado da média será **arredondado para o número inteiro mais próximo**, produzindo uma nota entre **0 e 5**.

Exemplo

Considere uma equipe com quatro integrantes:

Integrante	Pontuação
Membro 1	4
Membro 2	0 — ausente
Membro 3	5
Membro 4	3

A média será:

$$(4 + 0 + 5 + 3) \div 4 = 3,0$$

Portanto:

Nota da apresentação = 3 pontos

13.7. Composição da nota final

A nota final do trabalho será calculada pela soma das notas obtidas nas duas etapas:

Nota final = nota da entrega + nota da apresentação

em que:

- **nota da entrega:** valor entre 0 e 5;
- **nota da apresentação:** valor inteiro entre 0 e 5;
- **nota final:** valor entre 0 e 10.

Exemplo 1

Nota da entrega: **4,2**

Nota da apresentação: **3**

Nota final = 4,2 + 3 = 7,2

Exemplo 2

Nota da entrega: **4,8**

Nota da apresentação: **5**

Nota final = 4,8 + 5 = 9,8

Exemplo 3 — integrante ausente

Pontuações individuais da apresentação:

- Membro 1: **4**
- Membro 2: **0** — ausente
- Membro 3: **5**
- Membro 4: **3**

Média:

$$(4 + 0 + 5 + 3) \div 4 = 3,0$$

Nota da apresentação: **3**

Se a nota da entrega for **4,5**:

Nota final = 4,5 + 3 = 7,5

Nesse caso, a ausência de um integrante reduz a nota da apresentação de toda a equipe, pois sua pontuação zero participa do cálculo da média.

14. Sessão de apresentação e verificação individual

Durante a sessão, qualquer integrante poderá ser solicitado a:

- demonstrar uma funcionalidade;

- explicar uma classe, método ou função;
- descrever uma regra do jogo;
- justificar uma decisão de implementação;
- identificar onde determinado requisito foi implementado;
- explicar um trecho desenvolvido originalmente por outro integrante;
- explicar o processo de carregamento e desserialização dos dados JSON;
- explicar a conversão de valores do JSON para tipos enumerados;
- explicar a normalização utilizada para ignorar acentuação;
- explicar o cálculo das estatísticas;
- justificar a organização arquitetural adotada;
- localizar responsabilidades específicas dentro do projeto;
- modificar ou complementar um pequeno trecho da solução.

Todos os integrantes deverão possuir conhecimento geral e técnico da solução entregue.

A sessão não se destina apenas à exposição ou demonstração visual do aplicativo. Seu objetivo central é verificar o **domínio individual sobre a solução implementada**, utilizando esse desempenho para compor a nota coletiva da apresentação.

15. Observações finais

O objetivo da atividade não é apenas produzir uma versão visualmente funcional do Jogo da Forca.

A avaliação considerará também a capacidade da equipe de transformar regras conhecidas em uma solução de software:

- funcionalmente correta;
- adequadamente modelada;
- organizada;
- legível;
- responsiva;
- coerente com princípios de qualidade de software;
- tecnicamente justificável.

Decisões de interface e organização interna que não estejam explicitamente determinadas neste enunciado ficam sob responsabilidade da equipe, desde que não contrariem os requisitos apresentados.

O atendimento visual ou funcional aparente de um requisito não substitui a necessidade de uma implementação tecnicamente adequada e compatível com as regras estabelecidas neste enunciado.